

Уважаемый пользователь!

Обращаем ваше внимание, что система Антиплагиус отвечает на вопрос, является тот или иной фрагмент текста заимствованным или нет. Ответ на вопрос, является ли заимствованный фрагмент именно плагиатом, а не законной цитатой, система оставляет на ваше усмотрение.

Отчет о проверке № 10067925

Дата загрузки: 2026-06-25 17:42:15

Пользователь: anon_D4mtjimwAr@antiplagius.ru, ID: 10067925

Отчет предоставлен сервисом «Антиплагиат»
на сайте antiplagius.ru/

Информация о документе

№ документа: 10067925

Имя исходного файла:

Kursovoy_proekt_otchet_kolesov_ilya_dopolnenny.docx

Размер файла: 0.25 МБ

Размер текста: 29540

Слов в тексте: 3941

Число предложений: 175

Информация об отчете

Дата: 2026-06-25 17:42:15 - Последний готовый отчет

Оценка оригинальности: 100%

Заимствования: 0%

100.00%

0%

Источники:

Доля в тексте	Ссылка
---------------	--------

Информация о документе:

СОДЕРЖАНИЕ Введение Оглавление ВВЕДЕНИЕ 3 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПРОЕКТИРОВАНИЯ БАЗ ДАННЫХ ДЛЯ УЧЁТА СТУДЕНТОВ 6 1.1. Анализ предметной области и постановка задачи 6 1.2. Реляционная модель данных и принципы нормализации 7 1.3. Обзор систем управления базами данных и средств разработки 9 2. ПРАКТИЧЕСКАЯ РАЗРАБОТКА БАЗЫ ДАННЫХ УЧЁТА СТУДЕНТОВ 11 2.1. Проектирование структуры БД и ER-модели 11 2.2. Реализация таблиц, связей и ограничений целостности 12 2.3. Индексы и оптимизация запросов 14 2.4. Разработка представлений, функций, процедур и триггеров 15 2.5. Разграничение прав доступа и типовые запросы 18 2.6. Тестирование базы данных и анализ результатов 20 ЗАКЛЮЧЕНИЕ 25 СПИСОК ИСТОЧНИКОВ 27 ПРИЛОЖЕНИЕ А 28 ВВЕДЕНИЕ В условиях стремительной цифровизации сферы образования вопросы эффективного хранения, обработки и защиты информации приобретают особую значимость. Современное учебное заведение ежедневно оперирует значительными объёмами данных: сведениями о студентах, учебных группах, факультетах, преподавательском составе, изучаемых дисциплинах и результатах промежуточной и итоговой аттестации. Ведение подобного учёта вручную или с использованием электронных таблиц общего назначения сопряжено с целым рядом существенных недостатков - высокой вероятностью ошибок ввода, дублированием информации, нарушением её целостности, а также практически полным отсутствием механизмов разграничения доступа к конфиденциальным сведениям. Актуальность темы курсового проекта обусловлена необходимостью создания надёжного и масштабируемого инструмента, способного автоматизировать процессы учёта студентов и обеспечить достоверность хранимых данных. Применение специализированной реляционной базы данных позволяет устранить перечисленные проблемы за счёт строгой структуризации информации, реализации ограничений целостности, использования транзакционной модели обработки данных и встроенных средств безопасности. Степень разработанности темы. Теоретические основы реляционных баз данных были заложены в фундаментальных трудах Э. Кодда, предложившего реляционную модель, и развиты в работах К. Дейта. Вопросы нормализации, проектирования и оптимизации запросов подробно освещены в современной учебной и научной литературе. Вместе с тем конкретная предметная область - автоматизированный учёт студентов учебного заведения - требует самостоятельного проектного решения, учитывающего специфику бизнес-процессов конкретной организации. Объектом исследования выступает процесс учёта студентов в учебном заведении. Предметом исследования являются методы и средства проектирования реляционной базы данных, обеспечивающей автоматизацию данного учёта. Целью курсового проекта является разработка реляционной базы данных для учёта студентов учебного заведения, обеспечивающей надёжное хранение, эффективную обработку и защиту информации. Для достижения поставленной цели были сформулированы и решены

следующие задачи: - провести анализ предметной области и сформулировать требования к базе данных; - изучить теоретические основы реляционной модели и принципы нормализации; - обосновать выбор системы управления базами данных и инструментальных средств; - спроектировать логическую структуру базы данных и построить ER-модель; - реализовать таблицы, связи и ограничения целостности средствами языка SQL; - разработать представления, хранимые функции, процедуры и триггеры; - организовать разграничение прав доступа на основе ролевой модели; - провести тестирование базы данных и проанализировать полученные результаты. Методологическую основу работы составили методы системного анализа, структурного проектирования, нормализации отношений, а также экспериментальное тестирование разработанной базы данных на контрольном наборе данных. Практическая значимость проекта заключается в возможности применения разработанной базы данных в качестве серверной части информационной системы реального учебного заведения, а также в учебном процессе при изучении технологий проектирования баз данных. Курсовой проект состоит из введения, двух глав, заключения, списка источников и приложений. В первой главе рассматриваются теоретические основы проектирования баз данных, во второй - практическая реализация и тестирование разработанной системы.

1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПРОЕКТИРОВАНИЯ БАЗ ДАННЫХ ДЛЯ УЧЁТА СТУДЕНТОВ

1.1. Анализ предметной области и постановка задачи

Предметная область "учёт студентов учебного заведения" представляет собой совокупность взаимосвязанных объектов и процессов, охватывающих организационную структуру образовательного учреждения и сопровождение обучения каждого студента. Для корректного проектирования базы данных необходимо выделить ключевые сущности предметной области и определить характер связей между ними. В результате анализа были выделены следующие основные сущности: - факультеты - структурные подразделения учебного заведения, объединяющие группы родственных направлений подготовки; - учебные группы - объединения студентов одного курса и направления, относящиеся к конкретному факультету; - студенты - обучающиеся лица, являющиеся центральным объектом учёта; - дисциплины - учебные предметы, изучаемые студентами в рамках образовательной программы; - преподаватели - сотрудники, осуществляющие обучение и оценивание студентов; - оценки (успеваемость) - результаты аттестации студентов по дисциплинам. Между выделенными сущностями существуют связи различных типов. Связь "факультет - группа" имеет тип "один-ко-многим": один факультет включает несколько учебных групп, тогда как каждая группа принадлежит ровно одному факультету. Аналогичный тип связи характерен для пары "группа - студент". Особое место занимает связь между студентами и дисциплинами, имеющая тип "многие-ко-многим": один студент изучает множество дисциплин, а каждая дисциплина изучается множеством студентов. Подобная связь не может быть реализована напрямую в реляционной модели и требует введения связующей сущности - в данном проекте ею выступает таблица оценок, которая дополнительно фиксирует преподавателя, выставившего оценку, и дату аттестации. Постановка задачи. На основании проведённого анализа были сформулированы следующие функциональные требования к разрабатываемой базе данных: - обеспечение надёжного и структурированного хранения сведений обо всех выделенных сущностях; - гарантия целостности и непротиворечивости данных за счёт ограничений ссылочной целостности и контроля допустимых значений; - возможность формирования аналитических отчётов (средний балл студентов, успеваемость по дисциплинам, численность факультетов); - автоматизация типовых операций ввода данных с проверкой бизнес-правил; - разграничение прав доступа между категориями пользователей системы; - ведение журнала аудита критически важных операций. Перечисленные требования определили дальнейшие проектные решения, рассматриваемые во второй главе настоящей работы.

1.2. Реляционная модель данных и принципы нормализации

Реляционная модель данных, предложенная Э. Коддом в 1970 году, является на сегодняшний день доминирующей моделью представления данных в информационных системах. В её основе лежит математическое понятие отношения (relation), которое на логическом уровне представляется в виде двумерной таблицы. Строки таблицы соответствуют кортежам (записям), а столбцы - атрибутам (полям). Каждое отношение обладает уникальным первичным ключом (primary key), однозначно идентифицирующим каждую запись, а связи между отношениями реализуются посредством внешних ключей (foreign key). Ключевыми преимуществами реляционной модели являются строгая математическая основа, независимость логического представления данных от физического хранения, а также наличие универсального декларативного языка запросов SQL. Эти свойства обеспечивают гибкость, надёжность и переносимость разрабатываемых решений. Важнейшим этапом логического проектирования реляционной базы данных является нормализация - процесс приведения отношений к виду, исключающему избыточность данных и аномалии при их вставке, обновлении и удалении. Нормализация осуществляется последовательным приведением отношений к нормальным формам. Первая нормальная форма (1НФ) требует, чтобы все атрибуты отношения были атомарными, то есть неделимыми, а в таблице отсутствовали повторяющиеся группы. В разработанной базе данных это требование соблюдено: каждое поле хранит единственное значение определённого типа. Вторая нормальная форма (2НФ) требует, помимо соблюдения 1НФ, полной функциональной зависимости всех неключевых атрибутов от первичного ключа. Поскольку в проекте применяются простые первичные ключи на основе автоинкрементных суррогатных идентификаторов, частичные зависимости отсутствуют. Третья нормальная форма (3НФ) требует устранения транзитивных зависимостей, при которых неключевой атрибут зависит от другого неключевого атрибута. Для достижения 3НФ сведения о факультетах, группах, дисциплинах и преподавателях вынесены в самостоятельные таблицы и связаны с основными сущностями посредством внешних ключей. Так, например, информация о факультете не дублируется в каждой записи о студенте, а хранится единожды в таблице факультетов. Таким образом, спроектированная схема базы данных приведена к третьей нормальной форме, что гарантирует минимальную избыточность хранимой информации, исключает аномалии модификации и обеспечивает целостность данных.

1.3. Обзор систем управления базами данных и средств разработки

Для практической реализации спроектированной базы данных необходимо выбрать систему управления базами данных (СУБД) - программный комплекс, обеспечивающий создание, ведение и совместное использование баз данных множеством пользователей. На современном рынке представлено несколько широко распространённых реляционных СУБД, обладающих различными характеристиками. MySQL - свободно распространяемая реляционная СУБД с открытым исходным кодом,

отличающаяся высокой производительностью, простотой администрирования и широкой поддержкой со стороны сообщества разработчиков. Начиная с версии 8.0, MySQL поддерживает оконные функции, ролевую модель доступа, ограничения CHECK и расширенные возможности работы с транзакциями. PostgreSQL - объектно-реляционная СУБД, отличающаяся строгим соответствием стандарту SQL и развитыми средствами расширения функциональности. Обладает богатым набором типов данных и продвинутыми возможностями, однако более требовательна к ресурсам и квалификации администратора. Microsoft SQL Server и Oracle Database - мощные коммерческие СУБД корпоративного уровня, обеспечивающие высокую производительность и надёжность, однако требующие приобретения дорогостоящих лицензий, что ограничивает их применение в учебных и небольших проектах. По итогам сравнительного анализа в качестве СУБД для реализации проекта была выбрана MySQL версии 8.0. Выбор обоснован следующими факторами: свободное распространение и отсутствие лицензионных отчислений; высокая производительность при работе с типичными для учебного заведения объёмами данных; полная поддержка необходимых средств - транзакций, внешних ключей, ограничений целостности, хранимых функций и процедур, триггеров и ролевой модели доступа; широкая распространённость и наличие обширной документации. В качестве инструментальных средств разработки использовались язык структурированных запросов SQL, среда визуального проектирования и администрирования MySQL Workbench, а также система контроля версий Git для хранения исходного кода проекта. Для корректного хранения данных на русском языке во всей базе данных применена кодировка utf8mb4 с правилом сравнения utf8mb4_unicode_ci, в качестве подсистемы хранения выбран механизм InnoDB, поддерживающий транзакции и ограничения ссылочной целостности. Выводы по первой главе. В результате проведённого теоретического исследования была проанализирована предметная область учёта студентов, выделены ключевые сущности и связи между ними, сформулированы функциональные требования к базе данных. Рассмотрены основы реляционной модели и принципы нормализации, обоснован выбор третьей нормальной формы как целевой. Выполнен сравнительный анализ современных СУБД, по результатам которого в качестве платформы реализации выбрана MySQL 8.0. Полученные теоретические результаты служат основой для практической разработки, представленной во второй главе.

2. ПРАКТИЧЕСКАЯ РАЗРАБОТКА БАЗЫ ДАННЫХ УЧЁТА СТУДЕНТОВ

2.1. Проектирование структуры БД и ER-модели

На основе сформулированных в первой главе требований была спроектирована логическая структура базы данных, получившей название student_accounting. Структура включает семь таблиц, обеспечивающих хранение всей необходимой информации: - faculties - факультеты учебного заведения; - groups - учебные группы, относящиеся к факультетам; - students - студенты, входящие в состав групп; - subjects - учебные дисциплины; - teachers - преподаватели; - grades - оценки (связующая таблица "студент - дисциплина - преподаватель"); - audit_log - журнал аудита для регистрации операций удаления. Связи между таблицами реализованы посредством внешних ключей и образуют иерархическую структуру "факультет → группа → студент". Таблица grades реализует связь "многие-ко-многим" между студентами и дисциплинами, дополнительно фиксируя преподавателя и дату выставления оценки. Графическое представление логической структуры базы данных в виде ER-модели приведено на рисунке 1. Рисунок 1 - ER-модель базы данных "Учёт студентов" Как видно из приведённой диаграммы, каждая сущность снабжена первичным ключом, а связи между таблицами обозначены направленными линиями от родительской таблицы к дочерней. Подобная организация полностью соответствует требованиям третьей нормальной формы и обеспечивает отсутствие избыточности данных.

2.2. Реализация таблиц, связей и ограничений целостности

Создание структуры базы данных выполнено средствами языка определения данных (DDL) SQL. Перед созданием таблиц определяется сама база данных с указанием кодировки и правила сравнения символов: CREATE DATABASE student_accounting CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci; USE student_accounting; Рассмотрим реализацию ограничения целостности на примере центральной таблицы students, хранящей сведения о студентах: CREATE TABLE students (student_id INT AUTO_INCREMENT PRIMARY KEY, group_id INT NOT NULL, last_name VARCHAR(60) NOT NULL, first_name VARCHAR(60) NOT NULL, middle_name VARCHAR(60) NULL, birth_date DATE NOT NULL, gender ENUM('М','Ж') NOT NULL, email VARCHAR(120) UNIQUE, phone VARCHAR(20), admission_date DATE NOT NULL, status ENUM('учится','академ','отчислен','выпускник') NOT NULL DEFAULT 'учится', CONSTRAINT fk_student_group FOREIGN KEY (group_id) REFERENCES groups(group_id) ON UPDATE CASCADE ON DELETE RESTRICT) ENGINE=InnoDB; В приведённом фрагменте задействован широкий набор средств обеспечения целостности данных. Ограничение NOT NULL гарантирует обязательность заполнения ключевых полей (фамилии, имени, даты рождения, пола, даты поступления). Ограничение UNIQUE для поля email исключает дублирование адресов электронной почты. Перечисляемый тип ENUM ограничивает множество допустимых значений полей gender и status строго определённым набором, что предотвращает ввод некорректных данных. Внешний ключ fk_student_group связывает студента с учебной группой, при этом правило ON DELETE RESTRICT запрещает удаление группы, в которой числятся студенты, а ON UPDATE CASCADE обеспечивает автоматическое обновление ссылок при изменении идентификатора группы. Аналогичные ограничения применены и к другим таблицам базы данных. Так, в таблице groups ограничение CHECK (course BETWEEN 1 AND 6) контролирует допустимый диапазон номеров курсов, а в таблице grades ограничение CHECK (grade BETWEEN 2 AND 5) гарантирует, что выставляемые оценки находятся в установленных пределах пятибалльной системы: CREATE TABLE grades (grade_id INT AUTO_INCREMENT PRIMARY KEY, student_id INT NOT NULL, subject_id INT NOT NULL, teacher_id INT NOT NULL, grade TINYINT NOT NULL CHECK (grade BETWEEN 2 AND 5), grade_date DATE NOT NULL, CONSTRAINT fk_grade_student FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE CASCADE, CONSTRAINT fk_grade_subject FOREIGN KEY (subject_id) REFERENCES subjects(subject_id) ON DELETE RESTRICT, CONSTRAINT fk_grade_teacher FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id) ON DELETE RESTRICT) ENGINE=InnoDB; Особого внимания заслуживает выбор правил каскадного удаления. Для связи оценки со студентом установлено правило ON DELETE CASCADE: при удалении студента автоматически удаляются все связанные с ним оценки, что соответствует логике предметной области. В то же время удаление дисциплины или преподавателя, имеющих выставленные оценки, запрещено правилом RESTRICT, что предотвращает потерю исторических данных об успеваемости.

2.3. Индексы и оптимизация запросов

Для повышения производительности выполнения типовых запросов в базе данных созданы дополнительные индексы по полям,

наиболее часто используемым в условиях поиска и операциях соединения таблиц: `CREATE INDEX idx_student_lastname ON students(last_name); CREATE INDEX idx_grade_student ON grades(student_id); CREATE INDEX idx_grade_subject ON grades(subject_id);` Индекс `idx_student_lastname` ускоряет поиск студентов по фамилии - одну из наиболее частых операций при работе с базой данных. Индексы `idx_grade_student` и `idx_grade_subject` существенно повышают скорость выполнения операций соединения таблицы оценок с таблицами студентов и дисциплин при формировании аналитических отчетов о среднем балле. Следует отметить, что первичные ключи и поля с ограничением `UNIQUE` индексируются СУБД автоматически, поэтому дополнительное индексирование требуется только для прочих часто используемых полей. Чрезмерное количество индексов нежелательно, поскольку оно замедляет операции вставки и обновления данных, в связи с чем индексы созданы лишь для критически важных с точки зрения производительности полей.

2.4. Разработка представлений, функций, процедур и триггеров Для реализации бизнес-логики и автоматизации типовых операций в базе данных разработан комплекс программных объектов: представление, хранимая функция, хранимая процедура и триггер. Представление (VIEW). Представление `v_student_avg` инкапсулирует часто используемый запрос вычисления среднего балла каждого студента с указанием его группы и факультета, что упрощает построение аналитических отчетов: `CREATE OR REPLACE VIEW v_student_avg AS SELECT s.student_id, CONCAT(s.last_name, ' ', s.first_name, ' ', IFNULL(s.middle_name, '')) AS fio, g.name AS group_name, f.name AS faculty, ROUND(AVG(gr.grade), 2) AS avg_grade, COUNT(gr.grade_id) AS grades_count FROM students s JOIN groups g ON s.group_id = g.group_id JOIN faculties f ON g.faculty_id = f.faculty_id LEFT JOIN grades gr ON s.student_id = gr.student_id GROUP BY s.student_id, fio, g.name, f.name;` Использование оператора `LEFT JOIN` при соединении с таблицей оценок гарантирует включение в результат и тех студентов, которым еще не выставлены оценки. Функция `CONCAT` формирует полное имя студента, а функция `ROUND` округляет средний балл до двух знаков после запятой. Хранимая функция (FUNCTION). Функция `fn_student_age` вычисляет возраст студ